# Sprint 6 · Reto «Datos sucios, cliente impaciente» · en una página

> Versión corta para **trabajar con ella abierta**. La ficha completa (palabras nuevas, guía paso a paso, uso de la IA, errores frecuentes) sigue en Reto_Sprint_06.md; la plantilla del informe está en `informe_calidad_plantilla.md` y los datos en `reservas_sucio.csv`.

**El cliente.** TurisData os manda `reservas_sucio.csv`, exportado a mano de varios sistemas. El gerente quiere un primer modelo de cancelaciones «para ayer», pero no se fía de los datos: pide un informe de qué está mal y un proceso que limpie el fichero siempre igual.
**Qué entregáis (no cambia):** `informe_calidad.md`, `reto_S06.ipynb` y `README.md`. **Al final del sprint se suben a `sprint-06/` del repositorio del equipo: 3 ficheros del equipo + 1 autoevaluación y coevaluación por persona** (`autoevaluacion_nombre.md`).
**Tiempo en clase:** planning 10 min · reto 80 min (30 el día 1 y 50 el día 2) · entrega y review 35 min. Lo que no quepa, a casa o a la 6.ª hora.

### Tres niveles: el equipo decide hasta dónde llega

| Nivel | Qué hay que tener | Nota* |
|---|---|---|
| 🟢 **Mínimo** (todos los equipos) | Tabla de **al menos 6 defectos** (qué, dónde, cuántas filas, causa, tratamiento, por qué; los tres más graves destacados) · **una** función `limpiar_reservas` con docstring que devuelve la tabla limpia y un resumen de lo cambiado · el cuaderno corre con *Reiniciar y ejecutar todo* · README corto · subido a `sprint-06/` | 2 |
| 🟡 **Estándar** | `Pipeline` que imputa, escala y codifica **ajustando solo con train** · `DummyClassifier` y un modelo comparados con validación cruzada (media ± desviación) · 3 preguntas al cliente · IA responsable (país de origen) · README con versiones y uso de la IA · revisión con la rúbrica | 3 |
| 🔴 **Extra** | Tabla con 8–10 defectos · partición temporal en lugar de aleatoria, justificada · explicar o probar qué pasaría si se ajustara el escalado con todos los datos (fuga) | 4 |

*Orientativo; puntúa el docente con la rúbrica del curso (0–4 en cada criterio). El mínimo es una entrega válida y evaluable; la «Definition of Done» de la ficha completa describe el nivel **estándar**.

### Reparto sugerido de los 90 minutos

| Min | Qué | Se sabe que está hecho cuando… |
|---|---|---|
| 10 | Planning: roles (Scrum Master y responsable técnico) y tablero | Seis tarjetas con persona asignada |
| 30 | Inventario de defectos con `resumen_calidad` y gráficos; tabla del informe | Seis o más defectos con sus datos completos |
| 15 | `limpiar_reservas`, antes y después, y partición *train/test* | La función devuelve tabla y resumen sin tocar el original |
| 15 | *Pipeline*, *baseline* y validación cruzada | Hay una frase con media ± desviación, modelo frente a tonto |
| 20 | Informe, README y revisión con la rúbrica | Otra persona podría repetir el cuaderno con el README |

### Frases para el informe y el README

- «Hemos encontrado … defectos. El más grave es …, que afecta a … filas. Creemos que viene de … y lo tratamos con … porque …»
- «`limpiar_reservas` no toca el CSV original: devuelve una tabla nueva y un resumen con …»
- «Nuestro modelo obtiene un F1 medio de … ± …, frente a … del modelo tonto. Eso significa que …»
- «Límites: … Hemos usado IA para …; lo hemos revisado …»

### Antes de entregar

- **Archivos subidos en `sprint-06/`:** `informe_calidad.md` · `reto_S06.ipynb` · `README.md` · `autoevaluacion_nombre.md` (**uno por persona**) = 3 + N ficheros
- ☐ El cuaderno corre con *Reiniciar y ejecutar todo* · ☐ El informe recoge cada defecto con causa y tratamiento · ☐ Pipeline ajustado solo con *train*
- ☐ Baseline y modelo comparados · ☐ Sección «Revisión con la rúbrica» rellena en el informe · ☐ README con cómo ejecutar y versiones
- ☐ Autoevaluación y coevaluación: **una por persona**, subida a `sprint-06/`